Схема практичних занять з курсу
“Паралельні та розподілені обчислення”

Зміст

Зміст 1
Формат проведення лабораторних занять 2
Схема оцінювання 4
Завдання №1 5
Завдання №2 9
Завдання №3 11
Завдання №4 12
Завдання №5 14
Завдання № 6 17

Формат проведення лабораторних занять

Лабораторні заняття відбуваються в очному режимі (в аудиторії), якщо інше не регламентується деканатом факультету прикладної математики та інформатики.

Студенти виконують вказані у цих вказівках завдання, отримують необхідну допомогу від викладача та можливість спілкування з колегами та викладачем у форматі семінарів/лабораторних.

На початку лабораторних занять студентам пропонується виконати перше завдання суть якого полягає у виборі предметної області, функціональних і нефункціональних вимог та розробці початкової архітектури системи. Студент може запропонувати свою предметну область, або ж розглянути одну із пропонованих систем: наприклад, побудову сервіса для розподіленого пошуку у файлах, чи система векторизації текстів для LLM на базі Apache Kafka.

Пропоновані сервіси будуть розглядатися на лекціях детально “з нуля”.

Далі, студенти можуть працювати на лабораторних в одному із двох форматів, попередньо узгодивши із викладачем лабораторних:

1. *Стандартний формат*.
   Виконувати наперед сформульовані завдання викладачем; Формат передбачає виконання певної к-ті завдань, зазвичай 5-ть завдань. Умови завдань, терміни здачі і деталі оцінювання відбувається на практичних заняттях. Відвідування практичних занять є обов’язковим. Завдання здаються індивідуально на практичних заняттях у наперед встановлений термін. Завдання не здані в термін можуть бути здані за наявності вільного часу наприкінці семестру із максимальною оцінкою, що становить 50% від початкової

2. *Альтернативний формат*.
   Робота над власним проектом чи освоєння певного курсу (udemy, coursera etc) тематика якого цікава студенту та узгоджується із курсом “Проектування розподілених систем”. Студент пропонує власний поетапний план, узгоджує його із викладачем. Термін узгодження власного плану не має перевищувати 2-х тижнів. Визначення етапів і поетапна здача завдання є необхідною умовою (як мінімум два етапи).

Також, обидва формати передбачають підсумковий тест на основі матеріалу лекцій і практичних занять в кінці семестру.

Як стандартний, так і альтернативний формати, передбачають можливість **групового виконання**. Максимальний розмір команди – три

студенти. Необхідною умовою є обґрунтування доцільності і чіткого розподілу ролей і відповідальності кожного студента у команді.

Випадки академічної недоброчесності не толеруються.

Схема оцінювання

Індивідуальне завдання	**Максимальна оцінка (балів)**
Завдання №1	10
Завдання №2	10
Завдання №3	10
Завдання №4	25
Завдання №5 [або] Завдання №6	35
Тест / активність на парах	10
Загальна максимальна оцінка	100

Тест відбувається онлайн. Результат студент отримує одразу після проходження тесту. На наступному практичному занятті студент має право обговорити оцінку чи правильність відповідей.

Якщо студент отримав >50 балів на лабораторних заняттях, включаючи оцінку за тест, залік зараховується автоматично. У іншому випадку, студент має можливість отримати залік згідно залікової сесії деканату.

Завдання №1.

Тема	Налаштування робочого середовища. Вибір сервісу та пропозиція початкової архітектури
Максимальна оцінка	10 балів (+3 бонусні)
Термін виконання	Пропонується до 1-2 тижні. До узгодження на практичних

Завдання складається із двох частин:

1. Налаштувати робоче середовище; Вибрати технології та інструменти розробки.
2. Запропонувати початкову архітектуру системи, яка реалізує розподілений пошук у файлах (або альтернативної предметної області, див. нижче) або ж розподілену систему розрахунку TF-IDF на базі Apache Kafka. Проаналізувати та описати вимоги.
   Завдання не передбачає реалізацію чи програмування, а лише аналіз і спробу проектування архітектури системи на загальному рівні.

Студентам пропонується мова програмування *java* і пов’язані з нею технології розробки. Така рекомендація аргументується тим, що на лекціях будуть розглянуті практичні приклади переважно із використанням згаданих технологій.

Однак, як альтернативу, студент має право обрати іншу мову програмування та стек технологій, оцінивши його можливості для виконання завдань, із суттю яких лектор ознайомлює на вступній лекції чи на практичних заняттях.

Як предметна область, студенту пропонується розробити на вибір одну із двох систем:

1. Систему для розподіленого пошуку у файлах. Очікується, що такий сервіс є веб аплікацією, де користувач має змогу здійснювати повнотекстовий пошук певної фрази у файлах. Як результат, отримує список із найвідповідніших пошуковій фразі N документів, відсортовані за релевантністю (важливістю) відносно пошукової фрази. Потрібно передбачити масштабування сервіса, оскільки к-ть файлів і користувачів системи може зростати. Також, при розробці архітектури, необхідно передбачити як система працюватиме у випадку збоїв окремих компонент, забезпечуючи користувачу неперервну роботу без давнтаймів.
2. Модуль Data Ingestion для навчання LLM. Готові фреймворки (Spark/Flink) споживають забагато пам'яті для вашого специфічного заліза, тому прийнято рішення реалізувати власний

легкохідний механізм обробки потоку статей (наприклад, з Вікіпедії) для виділення ключових слів.

Впродовж семестру, студенти матимуть змогу разом із викладачем на лекціях поетапно аналізувати різні аспекти побудови розподіленої системи, покращити початкову архітектуру і реалізовувати систему для розподіленого пошуку “з нуля”, аналізувати аспекти використання Apache Kafka.

Альтернативно, студент може працювати над своїм проектом чи іншою предметною областю (див. нижче).

№	**Етап завдання і основні вимоги**	**Оцінка (балів)**
1	Обрати технології розробки. Інсталювати середовище розробки (IntelliJ) Налаштувати всі потрібні засоби (jdk, maven, docker).	1
2	Студент повинен продемонструвати вміння написати просту аплікації на вибраній мові програмування / тех. стеку, скомпілювати(інтерпретувати) і запустити код на локальному комп’ютері. Очікується: 1. Написати просту програму для демонстрації веб сервера, здатного опрацьовувати HTTP запити (повертати випадково згенеровану відповідь). 2. Вміти стартувати програму в докер контейнері.	2
3	Ознайомитись із можливостями, які надає Університет для віддаленого навчання (Udemy). Перевірити доступ	0.5
4	Ознайомитись із можливостями які надає Університет (на час проведення занять), щодо користування хмарними ресурсами зокрема Microsoft Azure. <u>Бонусні</u> (**+3**) бали за деплоймент розробленої тестової програми у хмарі Microsoft Azure із використанням студентської підписки. Сервіс повинен бути доступним в мережі інтернет і надавати URL за яким можна перевірити працездатність сервісу.	0.5
5	**Початкова архітектура системи, над яким студент працюватиме впродовж семестру.** Очікується: 1. Описати функціональні і нефункціональні вимоги до системи.	6

	2. Опис основних компонент системи і їх взаємодії. У вигляді тексту та діаграм, на розсуд студента. 3. Описати як планується задовільнити основні функціональні і нефункціональні вимоги. 4. Звіт. Форма і обсяг довільні. Подаються в узгоджений термін на занятті або завантажуються у відповідну папку в команді MS Teams. 5. Здатність презентувати звіт та відповісти на кілька запитань по змісту поданого звіту. Якщо студент обирає працювати над системою іншою від пропонованої (“Система для розподіленого пошуку у файлах” чи “Розробка розподіленої системи розрахунку TF-IDF на базі Apache Kafka”), необхідно також описати індивідуальний поетапний план роботи.	

Можливі приклади сервісів (альтернативні) для проектного завдання:

- **Система розподіленого пошуку у файлах (*Google search, Elasticsearch, Solr* тощо). Розглядається на лекціях.**
- Система для замовлення сервісу таксі (*Uber, Uklon, Bolt*)
- Соціальна мережа/месенджер (*Facebook, X (twitter), Quora*)
- Система для побудови вкороченого URL (*TinyURL*) [URL Shortening Service]
- Сервіс стрімінгового відео (*Youtube, Netflix*)
- Сервіс логування і моніторингу розподіленої системи (*kibana, Grafana*)
- Агрегатор даних [Web crawler / scraper] (наприклад коефіцієнтів на спортивні події, чи фінансовий аналіз ринку акцій тощо)
- Система типу *Резерв+* (інтеграція / API даних з різних реєстрів)
- Банківська система (проведення транзакцій, система для виявлення шахрайських дій тощо)
- <u>Свій варіант</u>

Корисні посилання:

- Як стартувати java програму в докер контейнері: https://www.docker.com/blog/how-i-built-my-first-containerized-java-web-application/
- Як задеплоїти аплікацію по ftp на Azure Cloud: https://learn.microsoft.com/en-us/azure/app-service/deploy-ftp?tabs=portal

Завдання №2.

Тема	Обрання лідера у розподіленій системі (або розподілене блокування)
Максимальна оцінка	10 балів
Термін виконання	Пропонується до 1-2 тижні. До узгодження на практичних

У завданні потрібно реалізувати обрання лідера у розподіленій системі із N вузлів. Студентам пропонується реалізувати спосіб запропонований і проаналізований на лекції, який базується на використанні Zookeeper.
Студент має право реалізувати інший підхід обговорений на лекції, за умови виконання вимог завдання.

Функціональні вимоги:

- система повинна реалізувати консенсус у розподіленій системі стосовно обрання лідера чи розподіленого блокування;

Нефункціональні вимоги:

- Система повинна бути стійкою до відмов. Тобто за непрацездатності лідера, новий лідер повинен автоматично переобиратися із наявних вузлів. Після відновлення, колишній лідер повинен приєднатися до система у звичайній ролі “worker”.
- Система повинна бути масштабованою. Тобто працювати швидко при великій кількості вузлів;
- Система повинна бути стійкої до одночасного підключення великої кількості вузлів (відомий “Herd Effect”).

№	**Етап завдання і основні вимоги**	**Оцінка (балів)**
1	Інсталювати Zookeeper. Допускається один із способів – локальна інсталяція, використання Docker контейнера чи деплоймент в MS Azure cloud. Zookeeper повинен розгортатись як кластер із трьох вузлів. Студент повинен розуміти і демонструвати етапи інсталяції і особливості налаштувань і параметрів Zookeeper.	2

	Студент повинен розуміти основні можливості Zookeeper CLI (переглянути список znode, створити/видалити znode тощо.)	
2	Вузли розподіленої системи. Написати код для вузла розподіленої системи. Використати java API для взаємодії із Zookeeper. Продемонструвати відмовостійкість Zookeeper як сервіса, і можливості автоматичного відновлення Java клієнта Zookeeper.	2
3	Реалізувати алгоритм обрання лідера із використанням Zookeeper, практичні аспекти якого розглядались на лекції.	2
4	Демонструвати обрання лідера/розподілене блокування у розподіленій системі і його відмовостійкості. Зазвичай, достатньо запустити вузол системи як окремий процес у терміналі на локальному компютері. Додаткові бонусні бали (макс. 10 б.) надаються , якщо студент здійснює деплоймент системи у хмарне середовище.	2
5	Звіт, у електронні чи паперовій формі. Звіти подаються в узгоджений термін на занятті або завантажуються у відповідну папку в команді Ms Teams. Звіт повинен містити короткий опис архітектурних підходів і використаних технологій. Обсяг звіту не регламентується. Здатність відповісти на кілька запитань по змісту поданого звіту.	2

Завдання №3.

Тема	Реєстр і виявлення сервісів (Service Registry and Service Discovery)
Максимальна оцінка	10 балів
Термін виконання	Пропонується 1 тиждень. До узгодження на практичних

У завданні потрібно реалізувати реєстр сервісів у розподіленій системи і механізм автоматичного виявлення сервісів.

Реєстр сервісів повинен бути стійким до відмов.

Студентам пропонується реалізувати спосіб запропонований і проаналізований на лекції, який базується на використанні Zookeeper.
Студент має право реалізувати інший підхід обговорений на лекції, за умови виконання вимог завдання.

№	**Етап завдання і основні вимоги**	**Оцінка (балів)**
1	Реалізувати реєстр сервісів на основі Zookeeper чи альтернативних технологій.	4
2	Реалізувати механізм автоматичного виявлення вузла у розподіленій системі та автоматичне виявлення падіння вузла і видалення непрацездатного сервіса із реєстру.	4
3	Звіт, у електронні чи паперовій формі. Звіти подаються в узгоджений термін на занятті або завантажуються у відповідну папку в команді Ms Teams. Звіт повинен містити короткий опис архітектурних підходів і використаних технологій. Обсяг звіту не регламентується. Здатність відповісти на кілька запитань по змісту поданого звіту.	2

Завдання №4.

Тема	Проектування і розробка системи для розподіленого пошуку у файлах
Максимальна оцінка	25 балів
Термін виконання	Пропонується до 4 тижнів. До узгодження на практичних

Студентам пропонується реалізувати розподілену систему пошуку у файлах на основі алгоритму TF-IDF, архітектура якої розглядалась на лекціях.
Функціональні вимоги:

- Користувач вводить пошукову фразу на веб сайті;
- Запит відправляється на фронтенд сервіс, який із використанням реєстру сервісів надсилає завдання до лідера кластера для пошуку документів;
- Лідер розподіляє роботу у кластері і, паралельно, із використанням доступних робочих вузлів здійснює пошук у файлах;
- Лідер надсилає відповідь фронтенд серсісу у вигляду списку із N документів за спаданням релевантності пошукової фрази і посилання на видповідну книгу (файл).
- Фронтенд серсіс перетворює отриману відповідь і надсилає її веб бравзеру.
- Користувач може відкрити документ у бравзері для читання, а також змінити параметри пошуку (фраза, к-ть документів, мінімальн а оцінка документа для потрапляння у список тощо).

Нефункціональні вимоги:

- Стійкість до відмов і масштабованість фронтенд сервіса;
- Стійкість до відмов і масштабованість кластера для розподіленого пошуку.

№	**Етап завдання і основні вимоги**	**Оцінка (балів)**
1	Веб сторінка	2
2	Створення фронтенд-сервіса	4
3	Реалізація алгоритму TF-IDF	2
4	Створення кластера для розподіленого	8
5	Демонстарція системи і виконання нефункціональних вимог	4

6	Звіт	3

Бонусними балами оцінюються покращення до запропонованої на лекції архітектури – наприклад індексування і зберігання частоти входження слів в документ у БД (mongodb, hazelcast, redis тощо), використання балансування навантаження тощо.

Завдання №5.

Тема	Підсистема Real-Time аналітики пошукових запитів
Максимальна оцінка	25 балів
Термін виконання	Пропонується до 3 тижнів. До узгодження на практичних

Постановка задачі

Передісторія: Ваша команда успішно запустила систему розподіленого пошуку файлів. Система стала популярною, і бізнес-замовник хоче бачити тренди в реальному часі: що шукають люди просто зараз і звідки вони приходять.

Завдання: Розробити мікросервіс (Analytics Worker), який підключається до системи пошуку асинхронно через **Apache Kafka**, обробляє потік запитів і генерує два звіти:

1. **Тренди:** Топ-10 найпопулярніших пошукових слів за ковзним вікном (наприклад, за останню 1 хвилину).
2. **Географія:** Топ-10 міст із найактивнішими користувачами за весь час роботи (або за довгий проміжок часу).

2. Архітектурні вимоги

А. Інтеграція (Producer)
Ви повинні модифікувати код вашого попереднього **Search Service** (або створити емулятор навантаження), щоб при кожному вхідному запиті він відправляв подію в топік Kafka search-queries.

- **Формат повідомлення (JSON):**

```
{
  "query_text": "distributed systems",
  "user_city": "Lviv",   // Емулюйте це поле (random list of cities)
  "timestamp": 1701234567890
}
```

- **Вимога:** Потік даних має бути інтенсивним (емуляція сотень запитів на секунду для тестування).

Б. Аналітика (Consumer / Stream Processor)
Реалізувати сервіс (Java/Python/Go), який:

1. Читає дані з топіка search-queries.

2. Виконує **агрегацію** даних (Count).
3. Періодично (наприклад, кожні 5-10 секунд) друкує в консоль або публікує в окремий топік analytics-results поточні рейтинги.

3. Технологічні обмеження та вказівки

Варіант реалізації (на вибір студента):

- **Рівень 1 (Low-level Consumer):** Використання стандартного KafkaConsumer.
  - *Виклик:* Вам доведеться вручну реалізувати буферизацію подій у пам'яті (наприклад, HashMap<Word, Count>) і механізм очищення старих даних для реалізації часового вікна.
  - *Рекомендація:* Для зберігання стану ("State Store") можна використати ConcurrentHashMap або вбудовану базу даних **RocksDB** (це буде плюсом, оскільки дозволяє відновити стан після перезапуску сервісу).
- **Рівень 2 (Stream Framework):** Використання **Kafka Streams** (Java) або **Faust** (Python).
  - Це спрощує задачу віконних функцій (.windowedBy(...)), але вимагає правильного налаштування топології (KStream -> KTable).

Вимога до точності:
Допускається "наближений" (approximate) підрахунок для Топ-10, якщо це дозволяє зекономити пам'ять (наприклад, алгоритм Count-Min Sketch), але для даного обсягу (25 балів) достатньо і точного підрахунку в пам'яті.

4. Критерії оцінювання (Макс. 25 балів)

Критерій	**Опис**	**Бали**
Інтеграція з Kafka	Продюсер коректно відправляє JSON-події. Консюмер їх зчитує. Правильно обрані ключі партиціювання (наприклад, по user_id або city для збереження порядку, або random для розподілу навантаження).	**5**
Реалізація Top-10 Queries (Windowing)	Реалізовано логіку "вікна" (Time Window). Система не показує запити, що були популярні годину тому, якщо зараз їх не шукають. Рейтинг оновлюється динамічно.	**10**
Реалізація Top-10 Cities (Global State)	Коректний підрахунок активності по містах. Агрегація відбувається коректно навіть при використанні декількох інстансів консюмера	**5**

Критерій	Опис	Бали
	(або обґрунтування, чому використовується лише один інстанс).	
Обробка та чистота коду, інфраструктура	Відсутність hardcode. Код розбитий на логічні модулі. Звіт містить приклад роботи (лог консолі або скріншот). Запуск кафка і сервісів локально чи у клауді.	**5**

5. Контрольні запитання для захисту

1. **State Management:** Де ваш сервіс зберігає проміжні лічильники (стан)? Що станеться з рейтингом Топ-10, якщо перезапустити ваш сервіс прямо зараз? (Втратяться дані чи відновляться з диска/RocksDB/Changelog топіка?)
2. **Time Semantics:** Який час ви використовуєте для вікна: *Event Time* (час створення події) чи *Processing Time* (час, коли сервер її отримав)? Чому?
3. **Late Events:** Що робить ваша система, якщо повідомлення з часовою міткою "5 хвилин тому" прийшло тільки зараз (через затримку мережі)? Воно потрапить у статистику?

Вимоги то інсталяції Apache Kafka:
Локальна, хмарна інсталяція чи використання Docker контейнера. Кафка кластер повинен складатися із 3-х вузлів. Версія Kafka не регламентується (можливе використання інсталяції що базується на Zookeeper чи Kraft)

Студент повинен вміти демонструвати працездатність kafka кластера, переглядати логи сервера, використовувати основні команди у терміналі для роботи з Apache Kafka.

Завдання № 6

Тема	Розподілена система векторизації текстів для LLM на базі Apache Kafka
Максимальна оцінка	25 балів
Термін виконання	Пропонується до 3 тижнів. До узгодження на практичних

Етап завдання і основні вимоги:

Завдання моделює реальні виклики при підготовці даних для навчання сучасних Large Language Models (LLM).

Ви працюєте у команді розробки LLM. Ваше завдання — побудувати пайплайн, який перетворює потік сирих текстових статей у числові вектори, придатні для навчання нейронної мережі.

Проблема: Класичні підходи (наприклад, sklearn.TfidfVectorizer) тримають весь словник у пам'яті однієї машини, що неможливо при обробці терабайтів даних.

Мета: Розробити розподілену систему, яка:

1. Формує **статичний словник** на основі історичних даних (Warm-up Phase).
2. Обробляє потік нових документів у реальному часі (Production Phase).
3. Генерує розріджені вектори (Sparse Vectors) на основі метрики **TF-IDF**.
4. Використовує **Apache Kafka** як єдиний засіб комунікації та збереження стану (State Persistence). **Використання зовнішніх БД (Postgres/Redis) не рекомендовано.**

Пропозиція Архітектура Системи
(може бути змінено студентом)

Система повинна працювати у двох режимах (фазах). Перемикання між режимами ініціюється спеціальною керуючою подією.

Фаза 1: Навчання (Warm-up / Learning)

1. **Вхід:** Історичний дамп даних (bootstrap-dataset).
2. **Дія:** Система рахує глобальну частоту слів. Вектори не генеруються.

3. **Результат:** Побудова словника (Vocabulary) та глобальної статистики (IDF Stats).

Фаза 2: Продакшн (Streaming / Production)

- **Вхід:** Потік нових статей (raw-articles).
- **Дія:** Воркери використовують сформований словник для перетворення тексту у вектори.
- **Результат:** Потік векторів у топіку ml-vectors.

Студент має реалізувати три компоненти, які взаємодіють через Kafka топіки. Система в цілому реалізує спрощений аналог MapReduce.

Компонент 1: Stats Reducer (Aggregator)

1. *Stateful* сервіс. Запускається в єдиному екземплярі (або з правильним партиціюванням).
2. Слухає топік *word-counts*.
3. Агрегує дані в пам'яті (рахує *DF* для кожного слова та глобальне *N*).
4. Періодично скидає абсолютні значення в топік *corpus-stats*.
5. Спеціальна логіка: При отриманні команди "Finalize Vocabulary" (або по завершенню фази 1):
    - Бере Топ-X найпопулярніших слів.
    - Присвоює їм ID від 0 до X.
    - Публікує словник у топік vocabulary-map.

Компонент 2: Document Processor (Worker)

Scalable сервіс. Запускається N екземплярів. Працює як State Machine з двома станами:

Стан А: LEARNING (Під час обробки bootstrap-dataset)

1. Розбиває текст на слова (Tokenization).
2. Відправляє унікальні слова документа в word-counts (лише 1 раз для кожного слова на документ).
3. Не генерує повідомлень у ml-vectors.
4.

Стан Б: PRODUCTION (Після сигналу з system-control)

1. Має в локальній пам'яті закешовані corpus-stats та vocabulary-map.
2. Читає raw-articles.

3. Для кожного слова:
    - Знаходить його ID у *vocabulary-map*. Якщо слова немає — ігнорує його.
    - Бере *DF* (Document Frequency) з кешу.
    - Рахує *TF* (Term Frequency) локально.
    - Обчислює Weight = *TF log(N / DF).*
    - Формує JSON Sparse Vector.
    - Відправляє у топік *ml-vectors*.

Компонент 3: Data Loader (Producer)

1. Читає дані, файли, статті із зовнішніх джерел. Для простоти – дозволяє завантажити текстовий файл по http API.
2. Публікує “сирі” тексти в топік raw-articles.
3. Вимога: Рівномірний розподіл навантаження (Round-robin).
4. Компонент Б: **Tokenizer & TF Calculator** (*Mapper*)
5. Вичитує повідом лення з `raw-articles`.

- Для масштабування і вискокодоступності - стартує к-ка екземплярів у спільній групі консюмерів (*Consumer Group*).
- Виконує токенізацію: рахує локальну частоту слів у документі (TF). А також, чистку від спецсимволів, видалення стоп-слів.
- Для кожного слова відправляє повідомлення `(word, doc_id, tf)` у топік `intermediate-words`.
    - **Важливо:** Ключем повідомлення має бути саме **слово**. Це гарантує, що всі входження слова "кібернетика" потраплять в одну й ту ж партицію, а отже — до одного й того ж Reducer-а.

3. Технічні обмеження та Вимоги

У даній лабораторній роботі не рекомендовано використовувати фреймворки Spark, Flink, Kafka Streams, Spring Cloud Stream. Дозволено використовувати лише "голий" Kafka Client (наприклад, `kafka-clients` для Java).

Топіки Kafka повинні мати мінімум 3 партиції. Система має запускатися мінімум у конфігурації: 1 Producer, 3 Mappers, 3 Reducers.

- Мова програмування: Java, Python, Go (на вибір).
- Інфраструктура: docker-compose, що піднімає Kafka, Zookeeper/Kraft та ваші сервіси.
- Заборонено: Використовувати будь-які бази даних, окрім Kafka.
- Заборонено: Змінювати ID слів після переходу у фазу Production.

Критерії оцінювання (Специфічні для Kafka)

Критерій	Деталізація
Kafka Engineering	Правильна конфігурація compacted топіків. Коректний вибір ключів (Partition Keys) для забезпечення порядку та локальності даних.
Архітектура (Phases)	Реалізовано механізм розігріву (Warm-up) та перемикання у Production через Control Topic. Система не генерує "сміттєві" вектори на старті.
ML-Ready Output	Вихідний вектор — це Sparse Vector з цілочисельними індексами (Static Vocabulary). Невідомі слова ігноруються. Формат відповідає JSON-схемі.
Algorithmic Logic	Правильний розрахунок DF (не плутати з Collection Frequency). Коректна математика TF-IDF.
Fault Tolerance	Воркери відновлюють свій стан (кеш статистики та словника) після рестарту, перечитуючи Compacted Topics.
Exactly-Once Processing	Як гарантувати, що слово не буде пораховано двічі, якщо Mapper впав після відправки повідомлення, але до коміту офсету?

Специфікація Kafka Топіків

Група А: Вхідні дані

1. bootstrap-dataset

   - **Тип:** Log (`cleanup.policy=delete`)
   - **Призначення:** Дані для фази навчання.
   - **Key:** `DocID` (String/UUID)
   - **Value:** `String` (Текст статті)
   -

2. raw-articles

   - **Тип:** Log
   - **Призначення:** Нові дані для обробки.
   - **Key:** `DocID` (String/UUID)
   - **Value:** `String` (Текст статті)

Група Б: Внутрішня комунікація та Стан

3. word-counts

- **Тип:** Log (Short retention, e.g., 1 hour)
- **Призначення:** Потік подій про знайдені слова.
- **Key:** `Word` (String) — *Важливо для партиціювання!*

- **Value:** `Long`
    - *Приклад 1:* `Key: "java", Value: 1`

4. corpus-stats

- **Тип: Compacted** (`cleanup.policy=compact`)
- **Призначення:** Глобальна база даних статистики.
- **Key:** `Word` (String) або `__GLOBAL_DOC_COUNT__`
- **Value:** `Long` (Абсолютне значення лічильника)
    - *Приклад:* `Key: "kafka", Value: 10500` (слово зустрічалося в 10500 документах)

5. vocabulary-map

- **Тип: Compacted** (`cleanup.policy=compact`)
- **Призначення:** Мапінг слів у цілі числа (Integer IDs) для ML.
- **Key:** `Word` (String)
- **Value:** `Integer` (Token ID)
    - *Приклад:* `Key: "apple", Value: 42`

6. system-control

- **Тип:** Broadcast (всі воркери читають цей топік).
- **Призначення:** Сигнали керування.
- **Key:** `Target` (e.g., "ALL")
- **Value:** `JSON` (Command)
    - *Приклад:* `{"command": "SWITCH_TO_PRODUCTION", "vocab_size": 10000}`

Група B: Вихідні дані

7. ml-vectors

- **Тип:** Log
- **Призначення:** Готовий датасет для навчання.
- **Key:** `DocID` (String)
- **Value:** `JSON` (**Sparse Vector**)

JSON

```
{
  "doc_id": "uuid-1234",
  "vector_size": 10000,
  "indices": [42, 105, 998],  // ID слів з vocabulary-map
  "values": [0.05, 0.12, 0.01] // TF-IDF ваги
}
```